EECE 7205: Fundamentals of Computer Engineering
In-Class Assignment #03

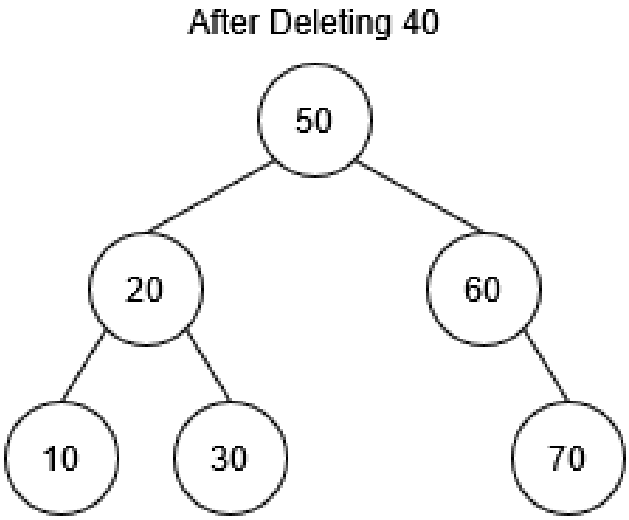
Katherine Tse
Email: tse.kat@northeastern.edu
NUID: 003936627
Due: October 18, 2026

1 Case 1: Deleting the Root (case1.cpp)

1.1 Question 1

- Nodes Visited: 60, 50
- Successor: 50
- Successor Parent: 60

1.2 Question 2



- Inorder Traversal: 10 20 30 50 60 70
- Preorder Traversal: 50 30 10 20 60 70
- The inorder traversal shows that rule B7 still holds because every value is still in increasing order from left to right.

Successor Code Output:

```
ktse@puter:~/EECE7205/InClassAssignment03_Tse$ ./case1
Before: 10 20 30 40 50 60 70    (root = 40)
After: 10 20 30 50 60 70        (root = 50)
```

1.3 Question 3

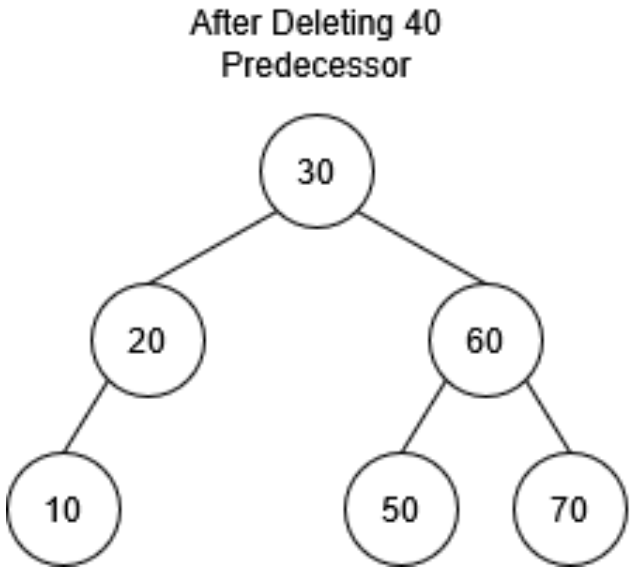
Predecessor Code:

```
// Case 1: delete the root (40). It has two children.
// 1. find the inorder predecessor: the largest value in the left
   subtree
Node* parent = root;
Node* pred = root->left; // go left once (20)
while (pred->right != nullptr) { // then right as far as possible
    parent = pred;
    pred = pred->right; // pred = 30, parent = 20
}
// 2. copy the predecessors's value into the root
root->data = pred->data; // root now holds 30
// 3. remove the predecessor node (it has no right child)
if (parent == root)
    parent->left = pred->left;
else
    parent->right = pred->left; // 20->right = NULL
delete pred;
```

Results

- New Root: 30
- The inorder output does not change because a valid binary tree's inorder output should still be a list of increasing values from left to right.

Tree T After



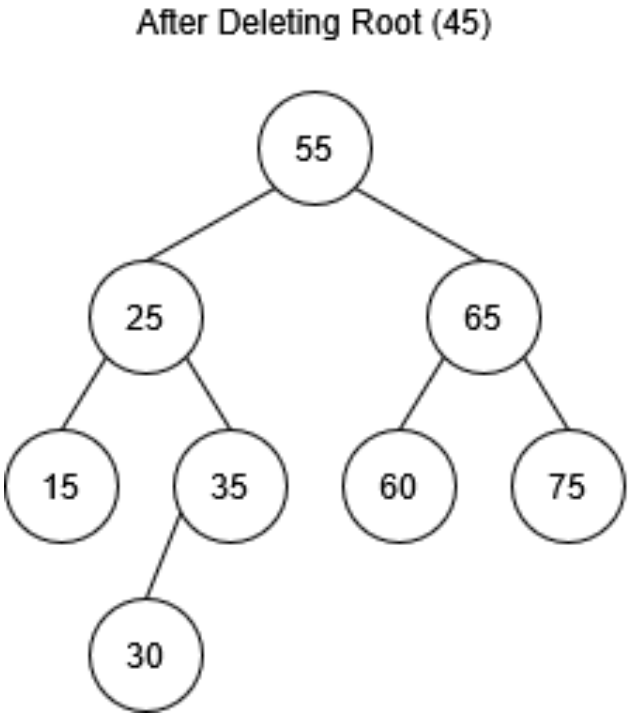
Predecessor Code Output:

```
ktse@puter:~/EECE7205/InClassAssignment03_Tse$ ./case1
Before: 10 20 30 40 50 60 70    (root = 40)
After: 10 20 30 50 60 70        (root = 30)
```

1.4 Question 4

- Successor Node: 55
- Rule B4 removed the successor node. It is not B3 because the successor had a right child node.

Tree P After:



Successor Code on Tree P Output:

```
ktse@puter:~/EECE7205/InClassAssignment03_Tse$ ./case1
Before: 15 25 30 35 45 55 60 65 75      (root = 45)
After: 15 25 30 35 55 60 65 75  (root = 55)
```

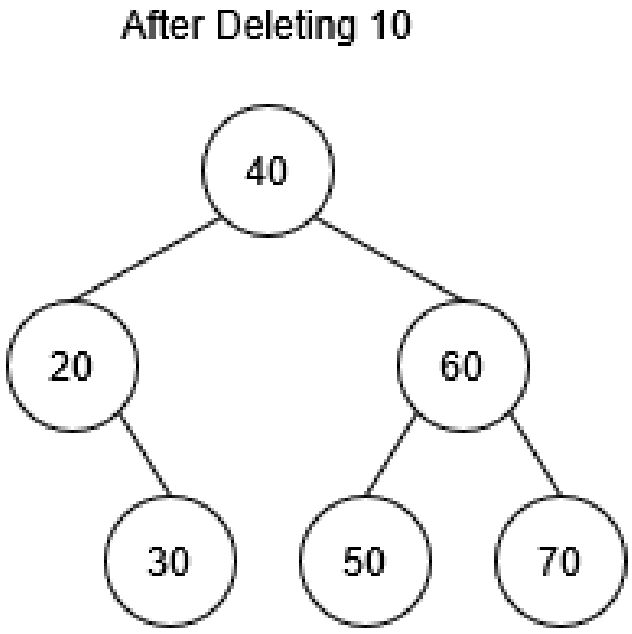
2 Case 2: Deleting a Leaf (case2.cpp)

2.1 Question 1

#	Parent	Curr	Check	Direction
1	NULL	40	$40 \neq 10$, $10 < 40$	Left
2	40	20	$20 \neq 10$, $10 < 20$	Left
3	20	10	$10 = 10$	Done

- Parent: 20
- Deleted Node: left child

Tree T After



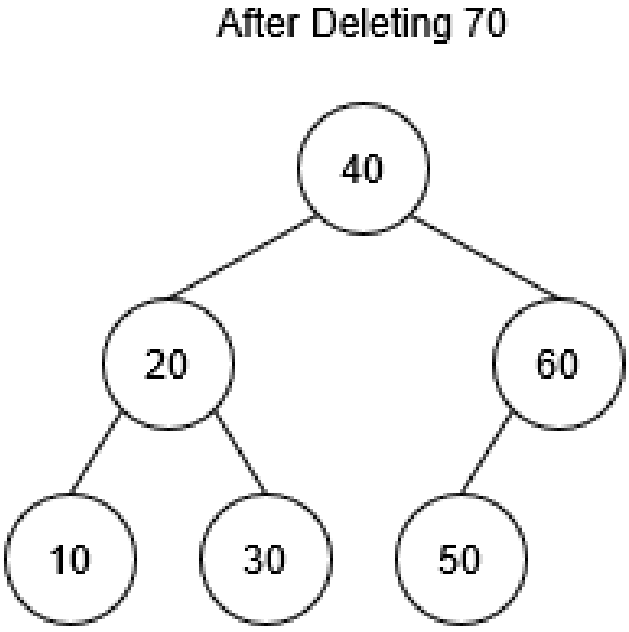
Delete 10 from Tree T Code Output:

```
ktse@puter:~/EECE7205/InClassAssignment03_Tse$ ./case2
Before: 10 20 30 40 50 60 70    (root = 40)
After: 20 30 40 50 60 70        (root = 40)
```

2.2 Question 2

- Pointer Set to NULL: 60 -> right child

Tree T After



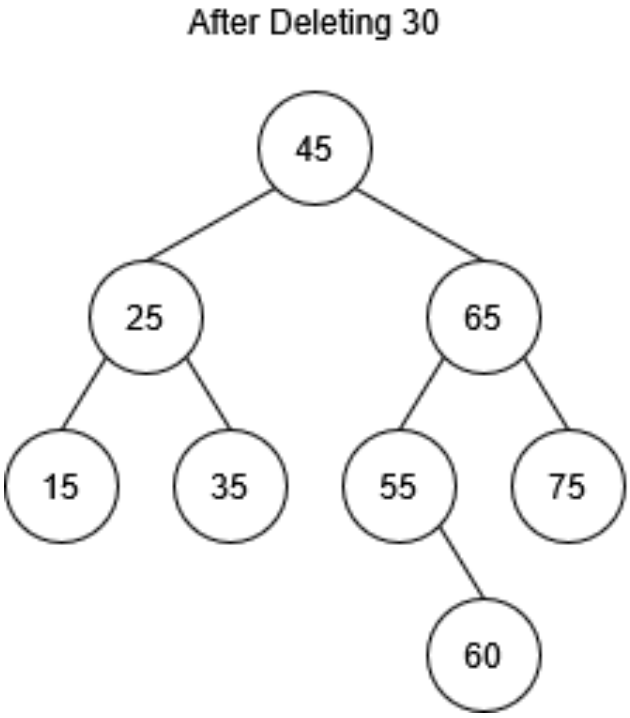
Delete 70 from Tree T Code Output:

```
ktse@puter:~/EECE7205/InClassAssignment03_Tse$ ./case2
Before: 10 20 30 40 50 60 70    (root = 40)
After: 10 20 30 40 50 60        (root = 40)
```

2.3 Question 3

- Leaves of P: 15, 30, 60, 75
- New Leaf: 35

Tree P After



Delete 30 from Tree P Code Output:

```
ktse@puter:~/EECE7205/InClassAssignment03_Tse$ ./case2
Before: 15 25 30 35 45 55 60 65 75      (root = 45)
After: 15 25 35 45 55 60 65 75  (root = 45)
```

2.4 Question 4

- At the worst case, there are 2h+1 key comparisons made. (Check if key = curr, if not, check if key < curr; only check if key = curr for target node)
- h for Tree T = 2
- h for Tree 10, 20, 30, 40, 50, 60, 70 (right skewed) = 6
- This tells us that the time to delete a leaf node is directly related to the height of the tree. As the height gets larger, the time to delete a leaf node will also get longer. The time could be represented as $O(h)$.

3 Case 3: Deleting a Node with One Child (case3.cpp)

3.1 Question 1

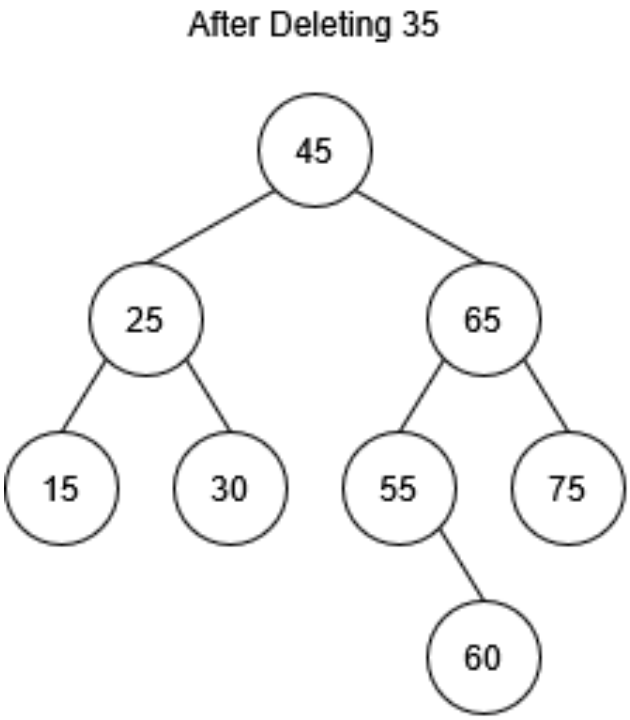
Single Child Nodes in Tree P

- 35, has left child
- 55, has right child

3.2 Question 2

- Parent: 25
- Child that moves up: 30
- Pointer changed: 25->right child

Tree P After



- Inorder traversal: 15 25 30 45 55 60 65 75

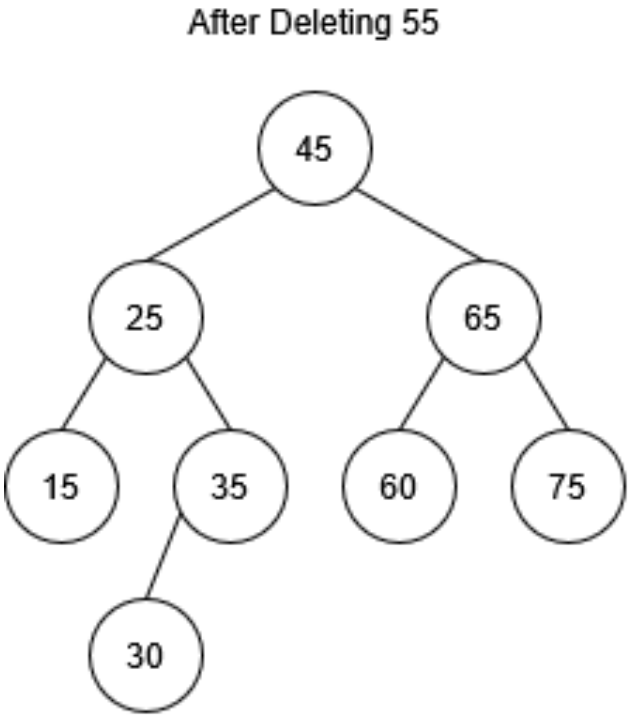
Delete 35 from Tree P Code Output:

```
ktse@puter:~/EECE7205/InClassAssignment03_Tse$ ./case3
Before: 15 25 30 35 45 55 60 65 75      (root = 45)
After: 15 25 30 45 55 60 65 75  (root = 45)
```

3.3 Question 3

- Parent: 65
- Child that moves up: 60
- Pointer changed: 65->left child

Tree P After



- Inorder Traversal: 15 25 30 35 45 60 65 75

Delete 55 from Tree P Code Output:

```
ktse@puter:~/EECE7205/InClassAssignment03_Tse$ ./case3
Before: 15 25 30 35 45 55 60 65 75      (root = 45)
After: 15 25 30 35 45 60 65 75  (root = 45)
```

The program uses parent->left instead of parent->right for this case because the node being deleted is the left child of the parent node. In the previous case, the node being deleted was the right child of the parent node and that is why parent->right was used.

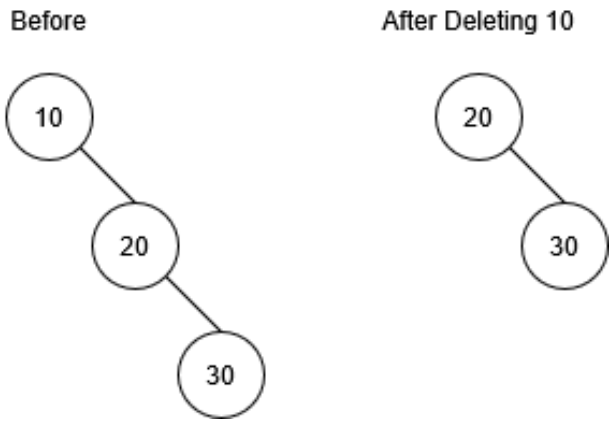
3.4 Question 4

- Rule B6 indicates that the root point itself must change because the root only has 1 child.
- New root: 20
- If the code is unchanged, the parent node for the root is set to a NULL pointer. This causes errors like a segmentation fault because a NULL pointer does not have a left or right child and therefore cannot be reassigned.

Updated Code to Fix Error:

```
// Case 3: delete a node with ONE child.
// 1. search for value, remembering its parent
Node* parent = nullptr;
Node* cur = root;
int key = 10;
while (cur != nullptr && cur->data != key) {
    parent = cur;
    if (key < cur->data) cur = cur->left;
    else                 cur = cur->right;
}
// 2. its only child takes its place
Node* child = (cur->left != nullptr) ? cur->left : cur->right;
if (cur == root)               root = child;
else if (parent->left == cur) parent->left = child;
else                           parent->right = child;
delete cur;
```

Tree After



Delete 10 from Tree Code Output

```
ktse@puter:~/EECE7205/InClassAssignment03_Tse$ ./case3
Before: 10 20 30        (root = 10)
After: 20 30    (root = 20)
```

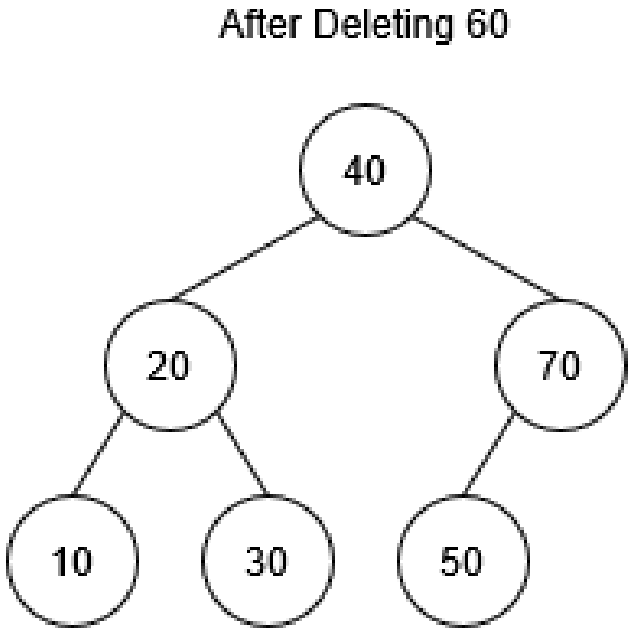
4 Case 4: Deleting a Node with Two Children (case4.cpp)

4.1 Question 1

- Successor: 70
- Parent: 60

- The first branch (parent->right = succ->right) runs because the successor's parent is the current node being deleted. To find the successor, you go right once first. Since the current node is still the parent, that one right move resulted in the successor node.

Tree T After



Delete 60 from Tree T Code Output

```
ktse@puter:~/EECE7205/InClassAssignment03_Tse$ ./case4
Before: 10 20 30 40 50 60 70    (root = 40)
After: 10 20 30 40 50 70        (root = 40)
```

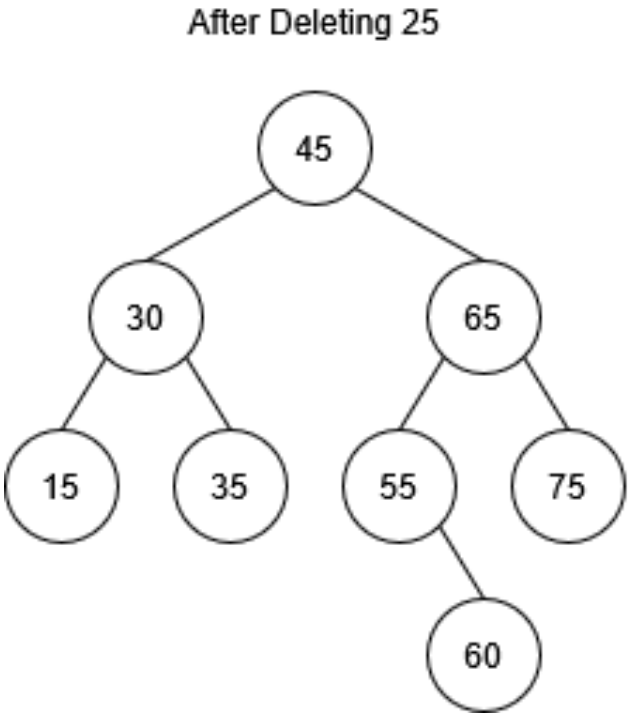
4.2 Question 2

The inorder successor of a node with two children will never have a left child because the successor will be the farthest left node possible of the right sub-tree of the node being deleted. If there were to be a left child, that would become the successor. If the successor could have a left child, Step 4 would result in the left child being severed from the tree. Step 4 only reconnects the right child of the successor node.

4.3 Question 3

- Successor: 30
- Parent: 35
- Pointer Changed: 35->left child

Tree P After:

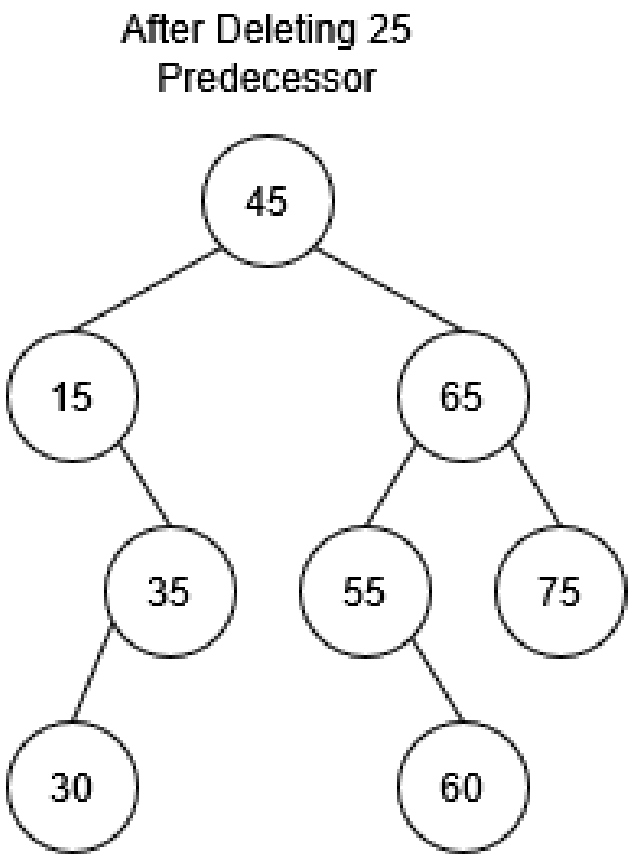


- Inorder Traversal: 15 30 35 45 55 60 65 75

Delete 25 from Tree P Code Output:

```
ktse@puter:~/EECE7205/InClassAssignment03_Tse$ ./case4
Before: 15 25 30 35 45 55 60 65 75      (root = 45)
After: 15 30 35 45 55 60 65 75  (root = 45)
```

4.4 Question 4



It is not the same tree as question 3, but both trees are valid binary search trees. For both trees, all nodes in the left sub-trees are less than the node's value, all nodes in the right sub-trees are less than the node's value, and this applies to all left and right sub-trees.

Predecessor Code:

```
// Case 4: delete a node with TWO children.
// 1. search for value
Node* cur = root;
int key = 25;
while (cur != nullptr && cur->data != key) {
    if (key < cur->data) cur = cur->left;
    else                 cur = cur->right;
}
// 2. find the inorder predecessor: left once, then right as far as
   possible
Node* parent = cur;
Node* pred = cur->left;
while (pred->right != nullptr) {
    parent = pred;
    pred = pred->right;
}
// 3. copy the predecessor's value into the node
```

```
17 cur->data = pred->data;
18 // 4. remove the predecessor node (it has no right child)
19 if (parent == cur)
20     parent->left = pred->left;
21 else
22     parent->right = pred->left;
23 delete pred;
```

Delete 25 from Tree P with Predecessor Code Output:

```
ktse@puter:~/EECE7205/InClassAssignment03_Tse$ ./case4
Before: 15 25 30 35 45 55 60 65 75      (root = 45)
After: 15 30 35 45 55 60 65 75  (root = 45)
```

5 Case 5: The Key Is Not in the Tree (case5.cpp)

5.1 Question 1

#	Curr	Check	Direction
1	40	$40 \neq 25$, $25 < 40$	Left
2	20	$20 \neq 25$, $25 < 20$	Right
3	30	$30 \neq 25$, $25 < 30$	Left
4	NULL	N/A	Done

- Comparisons = 6 (3 - data $\neq$ key, 3 - key $<$ data)
- Stops: 30->left child

5.2 Question 2

Key 5

#	Curr	Check	Direction
1	40	$40 \neq 5$, $5 < 40$	Left
2	20	$20 \neq 5$, $5 < 20$	Left
3	10	$10 \neq 5$, $5 < 10$	Left
4	NULL	N/A	Done

- Comparisons = 6 (3 - data $\neq$ key, 3 - key $<$ data)

- Stops: 10->left child

Key 45

#	Curr	Check	Direction
1	40	$40 \neq 45$, $45 < 40$	Right
2	60	$60 \neq 45$, $45 < 60$	Left
3	50	$50 \neq 45$, $45 < 50$	Left
4	NULL	N/A	Done

- Comparisons = 6 (3 - data $\neq$ key, 3 - key $<$ data)

- Stops: 50->left child

Key 75

#	Curr	Check	Direction
1	40	$40 \neq 75$, $75 < 40$	Right
2	60	$60 \neq 75$, $75 < 60$	Right
3	70	$70 \neq 75$, $75 < 70$	Right
4	NULL	N/A	Done

- Comparisons = 6 (3 - data $\neq$ key, 3 - key $<$ data)

- Stops: 70->Right child

All of the keys need the same number of comparisons. For all of them, a total of 2(h+1) comparisons occur. This makes sense as the algorithm must search nodes till a leaf node is found before concluding that the key is not in the tree.

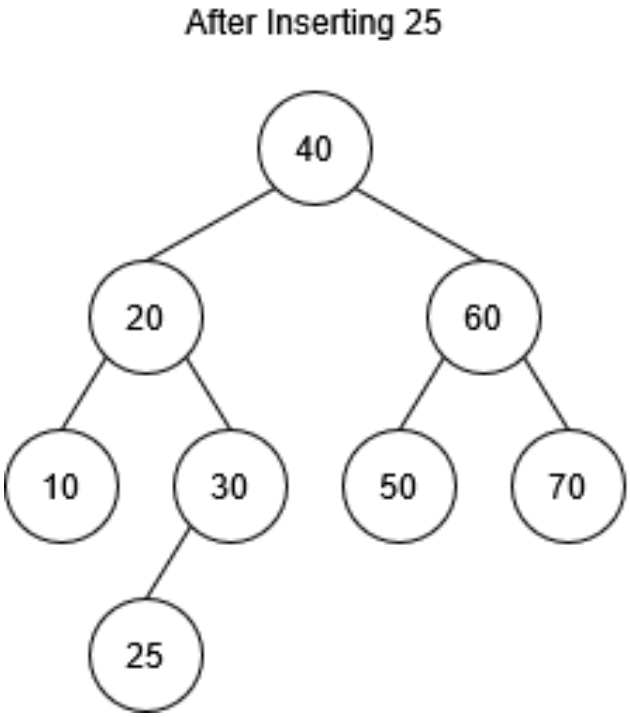
5.3 Question 3

Modified Case 5 to Insert When Not Found Code:

```
// Case 5: the key is not in the tree, insert instead.
Node* cur = root;
Node* parent = nullptr;
int key = 25;
while (cur != nullptr && cur->data != key) {
    parent = cur;
    if (key < cur->data) cur = cur->left;
    else                 cur = cur->right;
}
```

```
if (cur == nullptr)
    cout << key << " is not in the tree: nothing to delete" << endl;
    cout << "Inserting " << key << " at expected location..." <<
       endl;
    if (key < parent->data) parent->left = new Node(key);
    else                    parent->right = new Node(key);
```

Tree T After



Delete 25 from Tree T with Insertion When Not Found Output:

```
ktse@puter:~/EECE7205/InClassAssignment03_Tse$ ./case5
Before: 10 20 30 40 50 60 70    (root = 40)
25 is not in the tree: nothing to delete
Inserting 25 at expected location...
After: 10 20 25 30 40 50 60 70  (root = 40)
```

5.4 Question 4

- If the tree is empty (root = nullptr), nothing in the tree is changed when a delete is attempted.
- Rule B2 covers an empty tree because NULL is returned after the search.

Add Empty Tree Check to Case 5 Print Code:

```
cout << "Before: ";
printInorder(root);
if (root == nullptr) cout << "the tree is empty";
else                 cout << "\t(root = " << root->data << ")";
cout << endl;

// Case 5: the key is not in the tree.
Node* cur = root;
int key = 25;
while (cur != nullptr && cur->data != key) {
    if (key < cur->data) cur = cur->left;
    else                 cur = cur->right;
}
if (cur == nullptr)
    cout << key << " is not in the tree: nothing to delete" << endl;

cout << "After: ";
printInorder(root);
if (root == nullptr) cout << "the tree is empty";
else                 cout << "\t(root = " << root->data << ")";
cout << endl;
```

Delete 25 from Empty Tree Output:

```
ktse@puter:~/EECE7205/InClassAssignment03_Tse$ ./case5
Before: the tree is empty
25 is not in the tree: nothing to delete
After: the tree is empty
```

6 Bonus: One Function for Every Case (bonus.cpp)

Bonus Output:

```
ktse@puter:~/EECE7205/InClassAssignment03_Tse$ ./bonus
====Delete 40====
Before: 10 20 30 40 50 60 70    (root = 40)
After: 10 20 30 50 60 70        (root = 50)

====Delete 10====
Before: 10 20 30 50 60 70       (root = 50)
After: 20 30 50 60 70   (root = 50)

====Delete 60====
Before: 20 30 50 60 70  (root = 50)
After: 20 30 50 70      (root = 50)

====Delete 75====
Before: 20 30 50 70     (root = 50)
75 is not in the tree: nothing to delete
After: 20 30 50 70      (root = 50)
```

Time Complexity:

- Best Case: $O(1)$
- Average Case: $O(h)$
- Worst Case: $O(h)$

One Delete Function Code:

```
Node* deleteNode(Node* root, int key) {
    if (root == nullptr) {
        cout << "Tree is empty: nothing to delete" << endl;
        return root;
    }
    // search for value, remember it's parent
    Node* cur = root;
    Node* parent = nullptr;
    while (cur != nullptr && cur->data != key) {
        parent = cur;
        if (key < cur->data) cur = cur->left;
        else                 cur = cur->right;
```

```
13      }
14      if (cur == nullptr) {
15          // Case 5: the key is not in the tree.
16          cout << key << " is not in the tree: nothing to delete" <<
                endl;
17      } else if (cur == root) {
18          // Case 1: delete the root. It has two children.
19          // find the inorder successor: the smallest value in the
                right subtree
20          parent = root;
21          Node* succ = root->right; // go right once
22          while (succ->left != nullptr) { // then left as far as
                possible
23              parent = succ;
24              succ = succ->left;
25          }
26          // copy the successor's value into the root
27          root->data = succ->data;
28          // remove the successor node (it has no left child)
29          if (parent == root)
30              parent->right = succ->right;
31          else
32              parent->left = succ->right;
33          delete succ;
34      } else if ((cur->left == nullptr) && (cur->right == nullptr)) {
35          // Case 2: delete a leaf . It has no children.
36          // unlink it from its parent and free it
37          if (parent->left == cur) parent->left = nullptr;
38          else                     parent->right = nullptr;
39          delete cur;
40      } else if ((cur->left == nullptr) || (cur->right == nullptr)) {
41          // Case 3: delete a node with ONE child.
42          // its only child takes its place
43          Node* child = (cur->left != nullptr) ? cur->left : cur->
                right;
44          if (cur == root)              root = child;
45          else if (parent->left == cur) parent->left = child;
46          else                          parent->right = child;
47          delete cur;
48      } else {
49          // Case 4: delete a node with TWO children.
50          // find the inorder successor: right once, then left as far
                as possible
51          parent = cur;
52          Node* succ = cur->right;
53          while (succ->left != nullptr) {
54              parent = succ;
```

```
55              succ = succ->left;
56          }
57          // copy the successor's value into the node
58          cur->data = succ->data;
59          // remove the successor node (it has no left child)
60          if (parent == cur)
61              parent->right = succ->right;
62          else
63              parent->left = succ->right;
64          delete succ;
65      }
66
67      return root;
68  }
```